Summer Final Check-In

By Jason Lee

Purpose

- Create an MVP for a datapipeline for my mom's restaurant
  - At best, the product can go in to production
  - At worst, I'll brainstorm my next project

Table of Contents

- MVP Development
  - OCR
  - Google API Authentication
  - Google Drive + Google Sheets Services
  - ML unfinished

Restaurant Ticket Data Pipeline MVP



The goal of this data pipeline is to

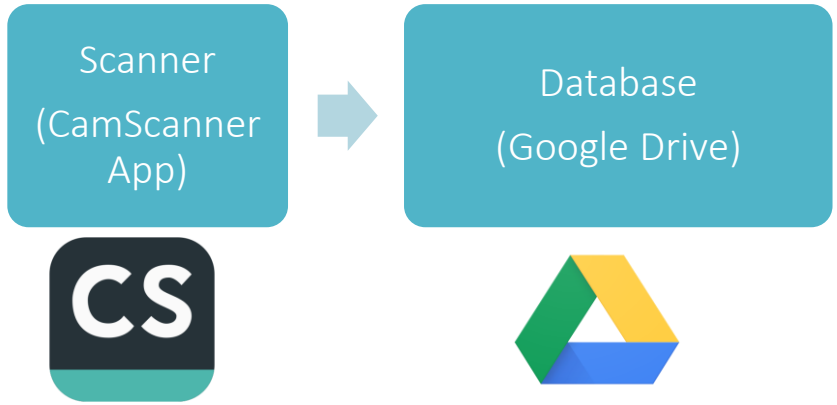
- Train a machine learning (ML) model to predict future customer orders and turn out
- Have a POS system that would do all the accounting for my mother.

Scanner: Upload to Google Drive

I would teach mom how to

- Scan all of the bills
- Share them to Google Drive in the restaurantocr@gmail.com account

She would scan the bills at the end of the day



Google API Services - Authentication

Before using the Google APIs, Oauth2.0, a standard protocol to obtain an access token, is required.

But Google's API authentication code is outdated, and the only solution to this problem was found in the Google API Python client documentation.

```
creds, _ = google.auth.default()

try:
  # create drive api client
  service = build("drive", "v3", credentials=creds,developerKey = '')

  file_id = real_file_id

  # pylint: disable=maybe-no-member
  request = service.files().get_media(fileId=file_id)
```

1. Enable the API in your Google Cloud console
2. Allow OAuth usage in "Branding"
3. Download credentials from "Clients"
4. Extract credentials with a configured scope to authenticate your program

```
def oauth2_authentication():
  flow = InstalledAppFlow.from_client_secrets_file(
      r'C:\Users\Jason Lee\OneDrive - University of South
      Florida\Personal\jupyter\Machine_Learning\$ML_PATH\CIS4900\Google\credential
      s.json',
      scopes=['https://www.googleapis.com/auth/drive','https://www.googleapis.com/
      auth/drive.readonly'])

  #Redirects user to local server to fetch credentials
  flow.run_local_server()

  creds = flow.credentials
  return creds
```

OCR – EasyOCR

- Sections of the image are read with “viewing rectangles” (visualized by the green rectangles around text)

- Text is displayed on the upper left of the rectangle

- No amount of fine-tuning will make EasyOCR accurate enough for production
  - “Rbh” is supposed to be “BBH” which stands for “Bun Bo Hue”
  - “bn” is supposed to be “BR” which stands for Bun Rieu
- Even if EasyOCR were reliable, one wrong order or miscalculation could ruin the accounting

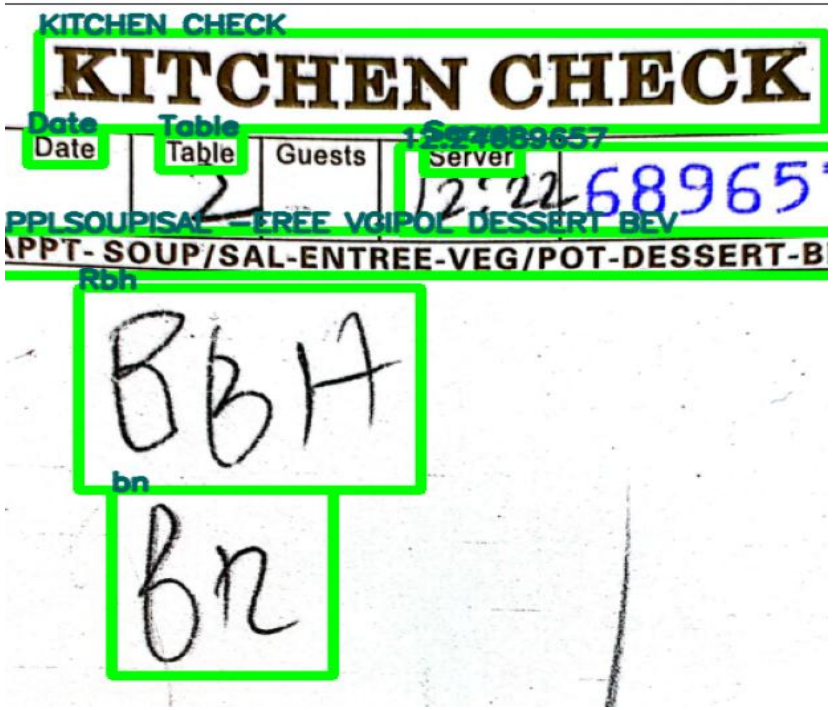
- Results are exported via a JSON file to the Google Sheets POS System

```
img = cv2.imread(IMAGE_PATH)
for x in range(len(result)-1):
    top_left = tuple(result[x][0][0])
    bottem_right = tuple(result[x][0][2])

    text = result[x][1]
    #0 is the nested list with the viewing rectangle
    #1 is the result of the OCR
    #2 is the confidence in that result
    font = cv2.FONT_HERSHEY_SIMPLEX

    img = cv2.rectangle(img,top_left,bottem_right,(0,255,0),5)
    img = cv2.putText(img,text,top_left,font,.6,(0,100,100),2,cv2.LINE_AA)

plt.figure(figsize=(10,10))
plt.imshow(img)
plt.show
```



Google Sheets Template

Date	Bill No	Quantity	Order	Table	Price	Modifications	Mod Price	Total	Total Bill	Order Type	Order Time	Payment Method
25-05-2025	689,951	2	*BBH*	4	$17.00		$2.00	$36.00	***$36.00***	On the...	2:00:00 PM	Cash
26-05-2025	689,952	1	*BBHXL*	3	$19.00			$19.00	***$19.00***	On-site		
27-05-2025	689,953	1	*CV*	3	$18.00			$18.00	***$18.00***	On-site		
28-05-2025	689,954	1	*BBH*	4	$17.00			$17.00	***$17.00***	On-site	2:00:00 PM	Cash
29-05-2025	689,955	2	*BM*	4	$17.00			$34.00	***$34.00***	To go (...		
30-05-2025	111,100	1	*GC*	4	$6.50			$6.50	***$6.50***	To go (...		Cash
31-05-2025	13	2	*BBH*	4	$17.00			$34.00	***$34.00***			
05/31/2025	999	2	*BBH*	6	$17.00			$34.00				
7/18/2025	999	2	*BBH*	6	$17.00	no feet	$3.00	$37.00	***$0.00***	To go	3:00 PM	Cash
31-05-2025	999	2	*BBH*	6		no feet	$3.00			To go	3:00 PM	Cash

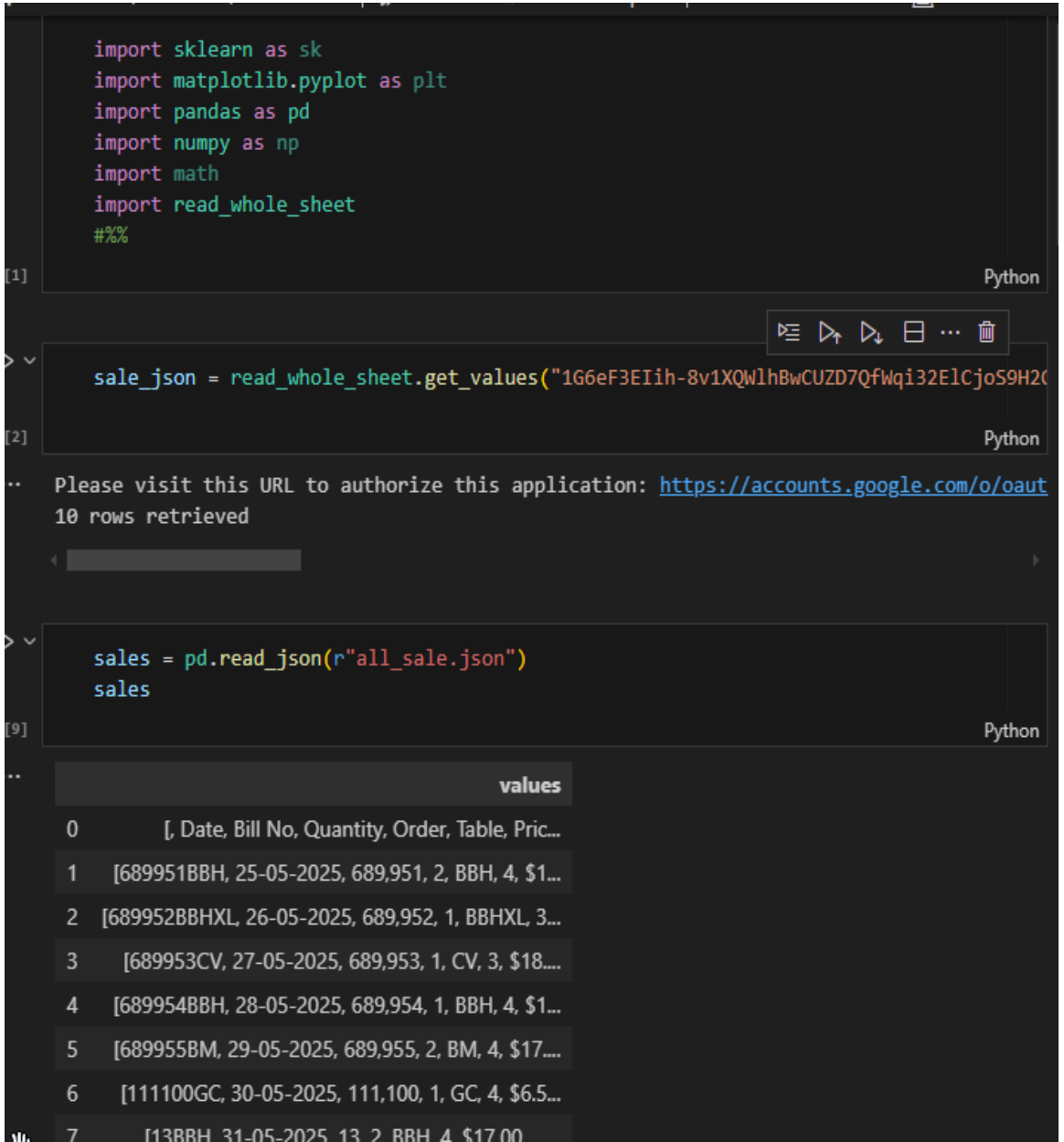
- After a bill is scanned with the OCR, the results are input into a point-of-sale (POS) database
- Such a database could show which order is correlated with order type, the time, payment methods, etc.
  - For example, Bun Bo Hue might be preferred for lunch rather than dinner!
- Template is taken and edited from: myeasytool

The ML Code

The Google Sheets .csv file is imported as a JSON file into a Jupyter notebook.

- At this point, I would be
  - Cleaning up the data.
    - Making categorical items like Orders into integers
    - Excluding the price and total columns
    - Scraping the internet for features
      - Weather conditions
      - Nearby events
- Train a logistic regression model (more on this later) with multiple categories

- Test the model on an unseen part of the dataset



Issues with the MVP

It wasn't until I got to this part that I realized

- There is little to no recorded features to base order predictions on
  - Whether someone orders a menu item is dependent on personal, hard-to-track variables.
  - Did they work out and need a meaty soup? What did their friend recommend they try? Have they eaten Vietnamese food before? Etc.
- Even if the MVP's purpose pivots to profit forecasting, more data is needed, which means hours 10s of hours of manual data input.
- The boss and the family don't need a ML model to know what customers order because we're collecting and interpreting the data itself